

INTRODUCTION

1.1 Introduction to the Domain

Artificial Intelligence (AI) has transformed various sectors including education. In the educational domain, AI enables predictive analytics to analyze student data and derive meaningful insights. The integration of Large Language Models (LLMs) such as Google Gemini with messaging platforms has enabled development of intelligent chatbots that understand context and provide helpful responses.

Educational Data Mining uses machine learning techniques to predict student performance and identify at-risk students. These predictions help institutions take timely action to support struggling students before they drop out.

1.2 Narrowing Down to Student Dropout Prediction

Student dropout is a critical problem in engineering colleges. Research shows approximately 14.8% of students dropout during their first year. This project develops a dropout prediction system that uses a WhatsApp chatbot to interact with students, collect behavioral data, and predict dropout risk.

The system connects at-risk students with faculty mentors who can provide guidance and support. An admin panel allows administrators to monitor all students, while a mentor portal enables mentors to track and help their assigned students.

1.3 Problems in Wide Sense

1. **Late Detection:** Institutions identify at-risk students only after academic failure, when intervention is too late.
2. **No Behavioral Data:** Traditional systems only track marks and attendance, missing stress and emotional indicators.
3. **Communication Gap:** Students hesitate to approach faculty due to fear of judgment.
4. **No Mentor Connection:** No systematic way to connect struggling students with supportive mentors.
5. **Manual Monitoring:** Administrators manually track students without any automated prediction system.

1.4 Solutions in Wide Sense

1. **Machine Learning Prediction:** ML algorithms can predict dropout probability from student data.
2. **WhatsApp Chatbot:** AI chatbot on WhatsApp can engage students and collect behavioral data naturally.
3. **Mentor Support System:** Connect at-risk students with faculty mentors for guidance.
4. **Admin Dashboard:** Centralized panel to monitor all students and predictions.
5. **Mentor Portal:** Dedicated interface for mentors to track assigned students.

1.5 Applications

1. Engineering Colleges
2. Universities
3. Online Learning Platforms
4. Vocational Training Institutes

RELATED WORK

Paper 1: Student Dropout Prediction using Machine Learning

Citation: F. Del Bonifro, M. Gabbrielli, G. Lisanti, and S. P. Zingaro, "Student Dropout Prediction," in Artificial Intelligence in Education, Springer, 2020, pp. 129-140.

Summary:

The paper predicts university student dropout using ML techniques on data from 15,000 students. Features included gender, age, high school marks, and course credits. Using SVM and Random Forest, they achieved 87% accuracy.

Limitations:

- Uses only academic data, no behavioral indicators.
- No intervention or support mechanism.
- No student-mentor connection system.

Paper 2: Conversational AI for Student Support

Citation: A. K. Sharma and R. Gupta, "Intelligent Student Support System using Conversational AI," International Journal of Educational Technology, vol. 18, 2022, pp. 234-248.

How it Addresses Paper 1's Limitations:

Introduces chatbot to capture behavioral data through conversations with sentiment analysis.

Summary:

Developed web-based chatbot using rule-based responses and VADER sentiment analysis. Achieved 82% prediction accuracy by combining academic and sentiment data.

Limitations:

- Web-based chatbot has low adoption (students don't visit portal).
- Rule-based responses feel robotic.
- No mentor support system.
- No admin monitoring dashboard.

Paper 3: LangChain-based Educational Chatbots

Citation: P. Kumar and S. Singh, "Building Context-Aware Educational Chatbots using LangChain," IEEE AIED, 2023, pp. 156-165.

How it Addresses Paper 2's Limitations:

Uses LangChain with LLMs for natural conversations with memory.

Summary:

Developed chatbot using LangChain with GPT. Natural language understanding improved student engagement significantly.

Scope for Modification:

1. Use WhatsApp instead of web portal for better adoption.
2. Add mentor support system for at-risk students.
3. Add admin dashboard for monitoring.
4. Add mentor portal for tracking assigned students.
5. Use Google Gemini (FREE) instead of OpenAI (PAID).

Our proposed system addresses all these gaps.

Comparison Table

Parameter	Paper 1	Paper 2	Paper 3	Proposed System
Platform	Offline	Web	Web	WhatsApp
Chatbot	No	Rule-based	LLM	LangChain + Gemini
Mentor Support	No	No	No	Yes
Admin Panel	No	No	No	Yes
Mentor Portal	No	No	No	Yes
Cost	-	Paid	Paid	FREE

PROBLEM DESCRIPTION

3.1 Statement of the Problem

Student dropout in engineering colleges is a significant problem. Students face academic difficulties, stress, and personal issues that lead to dropout. Currently, there is no system to:

- Regularly check on student well-being.
- Identify at-risk students early.
- Connect them with mentors.
- Monitor student status systematically.

3.2 Current Situation

- Institutions learn about dropout only when student submits application.
- No proactive communication with students.
- Faculty cannot identify struggling students early.
- No systematic mentor assignment for at-risk students.
- Manual tracking without any prediction system.

3.3 Impact

- 14.8% first-year dropout rate.
- Students lose career opportunities.
- Institutions lose reputation and revenue.
- Society loses skilled professionals.

3.4 Proposed Solution

This project proposes a system with four main components:

1. **WhatsApp Chatbot:** AI chatbot interacts with students, collects data, and identifies concerns.
2. **ML Prediction:** Predicts dropout probability from academic and behavioral data.
3. **Mentor-Student Chat:** Connects at-risk students with faculty mentors.
4. **Admin Panel & Mentor Portal:** Dashboards for monitoring and managing students.

3.5 Expected Goals

- Predict dropout risk with 80%+ accuracy.
- Enable regular student interaction via WhatsApp.
- Connect at-risk students with mentors.
- Provide monitoring dashboards for admin and mentors.

PROBLEM STATEMENT

Design and develop a Student Dropout Prediction System with WhatsApp Chatbot using LangChain and Google Gemini for student interaction, Machine Learning for risk prediction, Mentor-Student Chat for support, Admin Panel for monitoring, and Mentor Portal for tracking assigned students.

OBJECTIVES

Functional Objectives:

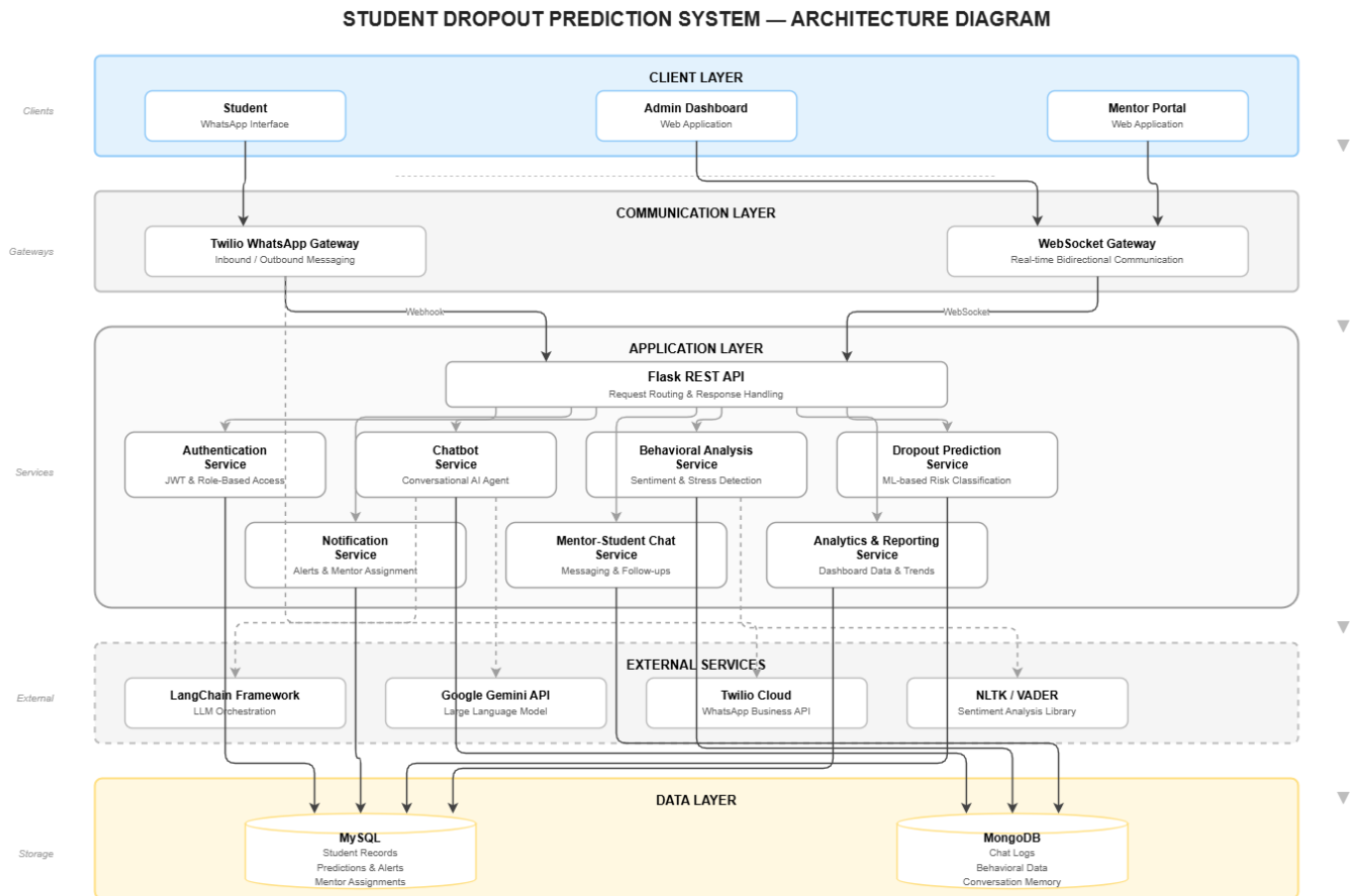
1. To develop a WhatsApp chatbot using **LangChain and Google Gemini API** for student interaction and data collection.
2. To implement a **Flask REST API backend** for handling authentication, chatbot processing, and system services.
3. To design an **Admin Dashboard and Mentor Portal** for monitoring students and supporting at-risk learners.

Performance Objective:

4. To achieve **80% or higher dropout prediction accuracy** using a Random Forest model.

METHODOLOGY

6.1 System Architecture



6.2 Module Descriptions

MODULE 1: DATA COLLECTION MODULE

Purpose:

To collect and store student academic data for dropout prediction.

Input:

- CSV file with student data (Roll No, Name, Gender, Age, 12th Marks, Semester Credits, etc.).
- Manual entry through admin panel.

Process:

1. Admin uploads CSV file through web interface.
2. System validates data (required fields, correct formats).
3. Data stored in MySQL database.
4. Students registered for WhatsApp chatbot.

Output:

- Student records in database.
- Students ready for chatbot interaction.

Technologies:

- Python, Pandas, Flask, MySQL

MODULE 2: WHATSAPP CHATBOT MODULE

Purpose:

To interact with students via WhatsApp, conduct check-ins, and collect behavioral data.

Input:

- Student messages via WhatsApp.
- Student profile from database.

Process:

1. Student sends message on WhatsApp.
2. Twilio receives message and sends to Flask server.
3. LangChain processes message with conversation memory.
4. Google Gemini generates appropriate response.
5. System extracts sentiment and behavioral features.
6. Response sent back to student via Twilio.

Output:

- Conversational responses to students.
- Behavioral data stored (sentiment, concerns mentioned).
- Alerts if student shows distress.

Technologies:

- Twilio WhatsApp API (Sandbox - FREE).
- LangChain Framework.
- Google Gemini API (FREE).
- Flask.
- MongoDB (for chat logs storage).

MODULE 3: BEHAVIORAL ANALYSIS MODULE

Purpose:

To analyze chatbot conversations and extract features for prediction.

Input:

- Chat messages from WhatsApp conversations.

Process:

1. Extract text from conversations.
2. Calculate sentiment score using VADER (-1 to +1).
3. Count stress-related keywords.
4. Calculate engagement metrics (message frequency, response rate).
5. Detect dropout signals (mentions of leaving, quitting).
6. Store features in database.

Output:

- Sentiment score.
- Stress level (Low/Medium/High).

- Engagement score.
- Dropout signal flag.

Features Extracted:

Feature	Description
avg_sentiment	Average sentiment of messages (-1 to +1).
stress_count	Number of stress keywords mentioned.
message_frequency	Messages per week.
engagement_score	How actively student responds.
dropout_signal	True if student mentioned leaving.

Technologies:

- VADER Sentiment Analysis
- Python, NLTK

MODULE 4: ML PREDICTION MODULE

Purpose:

To predict student dropout probability using machine learning.

Input:

- Academic features: Age, gender, marks, credits.
- Behavioral features: Sentiment, stress, engagement.

Process:

1. Combine academic and behavioral features.
2. Preprocess data (encode categories, scale numbers).
3. Apply Random Forest classifier.
4. Generate dropout probability (0-100%).
5. Classify risk level (LOW/MEDIUM/HIGH).

Output:

- Dropout probability percentage.
- Risk category (LOW < 30%, MEDIUM 30-70%, HIGH > 70%).

Risk Classification:

RISK LEVEL	PROBABILITY	ACTION
LOW	0% - 30%	Monitor
MEDIUM	31% - 70%	Watch
HIGH	71% - 100%	Assign

Technologies:

- Scikit-learn (Random Forest)
- Pandas, NumPy

MODULE 5: MENTOR-STUDENT CHAT MODULE

Purpose:

To enable real-time chat between students and assigned mentors.

Input:

- Student login.
- Mentor login.
- Chat messages.

Process:

1. Student logs in to chat portal.
2. Sees assigned mentor (if any).
3. Can send messages in real-time.
4. Mentor receives and responds.
5. All messages stored for reference.

Output:

- Real-time chat between student and mentor.
- Chat history stored in database.

Technologies:

- Socket.io (real-time messaging)
- Flask-SocketIO
- React.js (chat interface)
- MongoDB (chat storage)

MODULE 6: ADMIN PANEL MODULE

Purpose:

To provide administrators a dashboard to monitor all students and manage the system.

Input:

- Student data from database.
- Predictions from ML module.
- Alerts from chatbot.

Features:

1. Dashboard Overview:

- a. Total students
- b. Risk distribution (pie chart)
- c. Recent alerts

2. Student List:

- a. All students with risk scores
- b. Search and filter options
- c. Sort by risk level

3. Student Profile:

- a. Academic information
- b. Behavioral summary
- c. Prediction details
- d. Assigned mentor

4. Mentor Management:

- a. Add/remove mentors
- b. View mentor workload
- c. Assign mentor to student

5. Alerts:

- a. High-risk student alerts
- b. Crisis alerts
- c. Mark as resolved

Technologies:

- React.js
- Chart.js (for graphs)
- Material-UI (components)
- Flask REST API

MODULE 7: MENTOR PORTAL MODULE

Purpose:

To provide mentors a dedicated portal to view and support their assigned students.

Input:

- Mentor login.
- Assigned student data.

Features:

1. My Students:

- a. List of assigned students
- b. Risk scores and levels
- c. Quick view of concerns

2. Student Details:

- a. Academic information
- b. Chat summary (concerns raised)
- c. Behavioral trends
- d. Prediction details

3. Chat with Student:

- a. Direct chat link
- b. Chat history

4. Notes:

- a. Add notes about student
- b. Track follow-ups
- c. Record meeting outcomes

Technologies:

- React.js
- Flask REST API
- Material-UI

FACILITIES REQUIRED

7.1 Hardware Requirements

Component	Specification
Processor	Intel Core i5 or above
RAM	8 GB (16 GB recommended)
Storage	256 GB SSD
Internet	10 Mbps Broadband
Mobile	Android/iOS with WhatsApp

7.2 Software Requirements

Category	Technology	Purpose
IDE	VS Code	Development
Backend	Python 3.10+, Flask	Server
AI	LangChain, Google Gemini	Chatbot
Frontend	React.js	Admin/Mentor Portal
Real-time	Socket.io	Chat
Database	MySQL, MongoDB	Storage

7.3 Python Libraries

Library	Purpose
langchain	LLM framework
langchain-google-genai	Gemini integration
flask	Web server
flask-socketio	Real-time chat

twilio	WhatsApp API
scikit-learn	ML prediction
pandas, numpy	Data processing
vaderSentiment	Sentiment analysis
pymysql, pymongo	Database

7.4 Dataset

Source	Description
College Data	Student records from exam section
Google Forms	Survey responses
UCI Repository	Public Students Dataset

REFERENCES

- [1] F. Del Bonifro, M. Gabbrielli, G. Lisanti, and S. P. Zingaro, "Student Dropout Prediction," in *Artificial Intelligence in Education*, Springer, 2020, pp. 129-140.
- [2] C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis," *ICWSM*, 2014.
- [3] LangChain Documentation, <https://python.langchain.com/docs/>
- [4] Google Gemini API Documentation, <https://ai.google.dev/docs>
- [5] Twilio WhatsApp API Documentation, <https://www.twilio.com/docs/whatsapp>
- [6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, 2011.
- [7] Socket.IO Documentation, <https://socket.io/docs/>
- [8] React Documentation, <https://react.dev/>